

Esercizio

Implementare una chat di gruppo via multicast

- Ogni peer può sia mandare che ricevere messaggi
- Non c'è server centrale, ogni peer è sia server che client
- Usare un unico socket UDP per peer
- Usare select per gestire stdin + socket
- Ogni peer:
 - Si iscrive ad uno stesso gruppo (es. 239.1.2.3:5000)
 - Ascolta su socket UDP
 - Legge da tastiera (stdin)
 - Scrive a tutto il gruppo i messaggi letti da stdin
 - Scrive in stdout i messaggi che riceve via socket

Esercizio

Implementare una chat di gruppo via multicast

```
int sock;           // socket UDP multicast già creata, bindata, join al gruppo
struct sockaddr_in mcast_addr; // già inizializzata con group+port
char buf[1024];

while (1) {
    fd_set rfd;
    FD_ZERO(&rfd);
    FD_SET(STDIN_FILENO, &rfd);
    FD_SET(sock, &rfd);

    int maxfd = (sock > STDIN_FILENO ? sock : STDIN_FILENO) + 1;

    int ret = select(maxfd, &rfd, NULL, NULL, NULL);
    if (ret < 0) { perror("select"); break;}

    /* ... gestiamo stdin e socket ... */
}
```

Esercizio

Implementare una chat di gruppo via multicast

```
if (FD_ISSET(STDIN_FILENO, &rfdsets)) {
    if (!fgets(buf, sizeof(buf), stdin)) { printf("EOF, esco.\n"); break; }

    size_t len = strlen(buf);
    if (len > 0 && buf[len-1] == '\n') {
        buf[len-1] = '\0'; len--;
    }
    if (len == 0) {
        // riga vuota: niente da inviare
    } else {
        ssize_t n = sendto(sock, buf, len, 0,
                           (struct sockaddr *)&mcast_addr, sizeof(mcast_addr));
        if (n < 0) {
            perror("sendto"); break;
        }
    }
}
```

Esercizio

Implementare una chat di gruppo via multicast

```
if (FD_ISSET(sock, &rfdsets)) {
    struct sockaddr_in src;
    socklen_t srclen = sizeof(src);

    ssize_t n = recvfrom(sock, buf, sizeof(buf) - 1, 0,
                        (struct sockaddr *)&src, &srclen);
    if (n < 0) {
        perror("recvfrom"); break;
    }

    buf[n] = '\0';

    char ipstr[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &src.sin_addr, ipstr, sizeof(ipstr));

    printf("[%s] %s\n", ipstr, buf);
}
```

Comandi

- Comando: **netstat**

- **netstat [-t] [-all] [-p] [-n]**
 - elenca tutti i socket di rete del sistema (non riguarda i socket locali)
 - “-t” mostra solo i socket TCP, cioè quelli con famiglia=PF_INET e tipo=SOCK_STREAM
 - “-all” (oppure “-a”) mostra anche i socket in ascolto
 - “-p” specifica il pid del processo che ha creato ciascun socket
 - “-n” mostra gli indirizzi e le porte in formato numerico (invece di simbolico)
 - ad es., netstat -t -a -p -n mostra tutti i socket TCP aperti, indicando il PID del processo corrispondente e mostrando gli indirizzi in formato numerico

Comandi

- Comando: **netstat**

1) Mostrare tutte le connessioni TCP aperte

```
netstat -t
```

2) Mostrare anche quelle in ascolto (LISTEN)

```
netstat -t -a
```

3) Mostrare i processi (PID) associati ai socket

```
sudo netstat -t -a -p
```

4) Mostrare indirizzi e porte in formato numerico

```
netstat -t -a -n
```

5) Comando completo

```
sudo netstat -t -a -p -n
```

Comandi

- Comando: **netstat**

```
$ netstat -t
```

Active Internet connections (w/o servers)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	localhost:5200	localhost:54078	ESTABLISHED
tcp	0	0	localhost:54078	localhost:5200	ESTABLISHED

```
$ netstat -t -a -n
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	0.0.0.0:5200	0.0.0.0:*	LISTEN
tcp	0	0	127.0.0.1:5200	127.0.0.1:54078	ESTABLISHED
tcp	0	0	127.0.0.1:54078	127.0.0.1:5200	ESTABLISHED

Comandi

- Comando: **netstat**

1) Mostrare tutte le connessioni TCP aperte

```
netstat -t
```

2) Mostrare anche quelle in ascolto (LISTEN)

```
netstat -t -a
```

3) Mostrare i processi (PID) associati ai socket

```
sudo netstat -t -a -p
```

4) Mostrare indirizzi e porte in formato numerico

```
netstat -t -a -n
```

5) Comando completo

```
sudo netstat -t -a -p -n
```


Comandi

- Comando: **ss** (Socket Statistics)

- 1) Mostrare tutte le connessioni TCP aperte

```
ss -t
```

- 2) Mostrare anche quelle in ascolto (LISTEN)

```
ss -tln
```

- 3) Mostrare i processi (PID) associati ai socket

```
ss -tlnp
```

- 4) Mostrare indirizzi e porte in formato numerico

```
ss -tan
```

Comandi

- Comandi utili

`ip addr` (o `ip a`)

- mostra gli indirizzi IP della macchina Corrente
- es.: `ip addr | grep inet`
- sostituisce `ifconfig`

`nslookup <nome>`

- ottiene l'indirizzo IP da un nome di dominio
- es.: `nslookup www.unina.it`

`dig <nome>`

- alternativo più dettagliato per interrogare DNS

Comandi

- Comandi utili

ip

Utilità per visualizzare e configurare la rete (sostituisce ifconfig, route, arp).

- ip link: elenca tutte le interface (mostra flag, stato UP/DOWN, MAC add.)
- ip addr/a: mostra IPv4 e IPv6 assegnati
- ip route: verifica il gateway, Mostra la route di default e le reti locali

```
ip a s ens33          # dettagli dell'interfaccia ens33
ip link set ens33 down # disattiva interfaccia
ip link set ens33 up   # attiva interfaccia
```

Comandi

- Comandi utili

ip

Utilità per visualizzare e configurare la rete (sostituisce ifconfig, route, arp).

- ip link: elenca tutte le interface (mostra flag, stato UP/DOWN, MAC add.)

```
ip link show ens33
```

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500  
qdisc fq_codel state UP qlen 1000  
    link/ether 00:0c:29:57:95:d5 brd ff:ff:ff:ff:ff:ff
```

2: ID dell'interfaccia, ens33: nome dell'interfaccia

BROADCAST/MULTICAST: tipi di traffico supportati

UP: interfaccia attiva (software),

LOWER_UP: link fisico/virtuale attivo

mtu 1500: dimensione massima pacchetto, qdisc fq_codel: algoritmo coda pacchetti

state UP: stato operativo

qlen 1000: lunghezza coda TX

link/ether ...: MAC address

brd ...: indirizzo broadcast Ethernet

Comandi

- Comandi utili

ip

Utilità per visualizzare e configurare la rete (sostituisce ifconfig, route, arp).

- ip a: mostra gli indirizzi IP della macchina corrente

```
ip a show ens33
```

```
2: ens33: <UP,LOWER_UP> mtu 1500 qlen 1000  
    link/ether 00:0c:29:57:95:d5 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.29.128/24 brd 192.168.29.255  
    inet6 fe80::20c:29ff:fe57:95d5/64 scope link
```

- UP / LOWER_UP → attiva + link operativo
- mtu 1500 → dimensione pacchetto
- qlen 1000 → coda di trasmissione
- link/ether → MAC address
- brd → broadcast Ethernet e IPv4
- inet → indirizzo IPv4 + netmask
- inet6 → indirizzo IPv6 link-local

Comandi

- Comandi utili

ip

Utilità per visualizzare e configurare la rete (sostituisce ifconfig, route, arp).

- ip route: mostra le info di instradamento del sistema

```
ip route
```

```
default via 192.168.29.2 dev ens33 proto dhcp src  
192.168.29.128 metric 100  
192.168.29.0/24 dev ens33 proto kernel scope link src  
192.168.29.128 metric 100
```

Route predefinita: tutto il traffico destinato a Internet viene inviato al gateway 192.168.29.2 tramite ens33; la route è stata assegnata via DHCP, usa 192.168.29.128 come IP sorgente e ha priorità 100.

Route della rete locale: il traffico per la subnet 192.168.29.0/24 passa direttamente su ens33; è una route generata dal kernel, valida solo sul link locale, usa 192.168.29.128 come IP sorgente e ha priorità 100

Comandi

- Comandi utili

nslookup

Test DNS rapido e leggibile

- nslookup: è un comando per interrogare i server DNS

```
nslookup google.com
```

```
Server: 127.0.0.53
Address: 127.0.0.53#53
Non-authoritative answer:
Name: google.com
Address: 142.250.180.142
Name: google.com
Address: 2a00:1450:4002:403::200e
```

nslookup interroga il DNS locale (127.0.0.53, systemd-resolved) e restituisce gli indirizzi IPv4/IPv6 del dominio. La risposta è "non autorevole" perché proviene da un DNS ricorsivo e non dal Name Server ufficiale.

Comandi

- Comandi utili

nslookup

Test DNS rapido e leggibile

- nslookup: è un comando per interrogare i server DNS

```
nslookup -type=NS google.com  
nslookup google.com ns2.google.com
```

```
server: ns2.google.com  
Address: 216.239.34.10#53
```

```
Name: google.com  
Address: 74.125.29.138  
Name: google.com  
Address: 74.125.29.139  
Name: google.com  
Address: 74.125.29.113
```

Prima cerca il DNS autorevole per google.com, poi lo interroga

Comandi

- Comandi utili

dig (*Domain Information Groper*)

- dig: tool avanzato per interrogare il DNS
di default interroga il DNS configurato sul sistema

dig google.com	standard
dig +short google.com	risposta compatta
dig google.com @8.8.8.8	DNS specifico
dig +trace google.com	traccia completa
dig -x IP	reverse lookup

`dig google.com` interroga il resolver locale (es., 127.0.0.53), che usa la ricorsione del DNS esterno. La risposta è "NON autorevole" se proviene dalla cache del ricorsivo, non dal DNS ufficiale di Google. Viene restituito il record A con il TTL.

`dig +trace dominio` esegue una risoluzione DNS completa senza usare cache o resolver locali. Mostra tutti i passaggi effettuati: dai **root servers**, ai **server del TLD**, fino ai **DNS autorevoli** del dominio. È lo strumento per capire la catena DNS.

Comandi

- Comandi utili

dig (*Domain Information Groper*)

- dig: tool avanzato per interrogare il DNS
di default interroga il DNS configurato sul sistema

dig google.com	standard
dig +short google.com	risposta compatta
dig google.com @8.8.8.8	DNS specifico
dig +trace google.com	traccia completa
dig -x IP	reverse lookup

dig -x IP esegue un lookup inverso e riceve un PTR (pointer record)

;; ANSWER SECTION:

142.180.250.142.in-addr.arpa. 5 IN

PTR

mil04s43-in-f14.1e100.net.

Telnet & Netcat

■ Comandi utili

- `telnet <indirizzo> <porta>`
 - crea un socket e lo collega all'indirizzo remoto dato
 - esempio: “telnet www.unina.it 80” si mette in comunicazione con il server http dell'università
 - quello che si digita da terminale viene mandato sul socket
 - quello che proviene dal socket viene stampato su stdout
 - telnet può quindi fungere da “client generico”

Lanciare server su un terminale in ascolto su 5201

```
telnet localhost 5201
ciao
→ ciao
```

nc (netcat)

```
nc localhost 5200
```

Nome Dominio

- DNS

- Ad un host può venir assegnato un nome di dominio (*domain name*), come www.unina.it
- Il servizio di rete chiamato DNS (*Domain Name Service*) converte i nomi di dominio in indirizzi IP
 - www.unina.it → 143.225.5.3
 - questa conversione prende il nome di “risoluzione del nome”, dall'inglese “domain name resolution”
- Dalla shell, il comando nslookup esegue la conversione
 - nslookup <nome>
 - esempio: nslookup www.unina.it

Nome Dominio

- Recuperare l'indirizzo del dominio

```
struct addrinfo hints = {0}, *res;  
hints.ai_family = AF_INET;    // solo IPv4  
getaddrinfo(argv[1], NULL, &hints, &res);  
struct sockaddr_in *a = (struct sockaddr_in*)res->ai_addr;  
printf("IP: %s\n", inet_ntoa(a->sin_addr));  
freeaddrinfo(res);
```

`getaddrinfo()` → risolve un nome in una lista di strutture `addrinfo`
`inet_ntop()` → converte indirizzi binari in stringhe leggibili

Nome Dominio

- Recuperare l'indirizzo del dominio

getaddrinfo() è la funzione moderna e portabile per risolvere nomi simbolici (es. `www.example.com`) in indirizzi IP (IPv4/IPv6) e ottenere i parametri necessari per aprire una socket.

Risolve un nome in uno o più indirizzi IP

Supporta IPv4 e IPv6 senza differenze per il programmatore

Restituisce una lista di struct `addrinfo` pronte per `socket()` e `connect()`

```
struct addrinfo hints, *res;  
memset(&hints, 0, sizeof(hints));  
hints.ai_socktype = SOCK_STREAM; // TCP  
  
getaddrinfo("www.example.com", "80", &hints, &res);  
  
// res contiene indirizzi pronti per socket() e connect()
```

Nome Dominio

- Recuperare l'indirizzo del dominio

Utilizzo con scorrimento degli indirizzi

```
struct addrinfo hints, *res, *p;
memset(&hints, 0, sizeof(hints));
hints.ai_socktype = SOCK_STREAM;
hints.ai_family   = AF_UNSPEC; // IPv4 e IPv6

getaddrinfo("www.example.com", "80", &hints, &res);

int sock;
for (p = res; p != NULL; p = p->ai_next) {
    sock = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (sock < 0) continue;

    if (connect(sock, p->ai_addr, p->ai_addrlen) == 0) {
        printf("Connesso!\n"); break; }
    close(sock);
}
if (p == NULL) {
    fprintf(stderr, "Nessun indirizzo si è connesso\n"); exit(1);}

freeaddrinfo(res);
```

Nome Dominio

- Recuperare l'indirizzo del dominio

```
#define _POSIX_C_SOURCE 200112L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <unistd.h>
```

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <hostname>\n", argv[0]);
        exit(1);
    }
    struct addrinfo hints, *res, *p;
    char host[INET6_ADDRSTRLEN];
    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_UNSPEC; // IPv4 + IPv6
    hints.ai_socktype = SOCK_STREAM; // oppure 0 per qualsiasi
```


Nome Dominio

- Recuperare l'indirizzo del dominio

```
int err = getaddrinfo(argv[1], NULL, &hints, &res);
if (err != 0) {
    fprintf(stderr, "Errore: %s\n", gai_strerror(err));
    exit(1);
}
for (p = res; p != NULL; p = p->ai_next) {
    void *addr;
    const char *ver;

    if (p->ai_family == AF_INET) {
        struct sockaddr_in *ipv4 = (struct sockaddr_in *)p->ai_addr;
        addr = &(ipv4->sin_addr);
        ver = "IPv4";
    } else if (p->ai_family == AF_INET6) {
        struct sockaddr_in6 *ipv6 = (struct sockaddr_in6 *)p->ai_addr;
        addr = &(ipv6->sin6_addr);
        ver = "IPv6";
    } else { continue; }
    inet_ntop(p->ai_family, addr, host, sizeof(host));
    printf("%s: %s\n", ver, host);
}
freeaddrinfo(res);
return 0;
}
```

Esercizio

Implementare un mini client WHOIS in C

Scrivere un programma C che:

- accetta un argomento da linea di comando (es. example.com)
- risolve il nome
 - whois.iana.org o whois.verisign-grs.com usando getaddrinfo()
- apre una connessione TCP verso porta 43
- invia il nome richiesto seguito da \r\n
- riceve e stampa la risposta del server WHOIS
- chiude la connessione

Esercizio

Implementare un mini client WHOIS in C

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <netdb.h>
#include <arpa/inet.h>

int main(int argc, char *argv[]) {
    if (argc < 2) return 1;

    struct addrinfo hints, *res;
    memset(&hints, 0, sizeof(hints));
    hints.ai_socktype = SOCK_STREAM;
    getaddrinfo("whois.iana.org", "43", &hints, &res);
```

Esercizio

Implementare un mini client WHOIS in C

```
int sock = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    connect(sock, res->ai_addr, res->ai_addrlen);
    freeaddrinfo(res);
    char query[256];
    snprintf(query, sizeof(query), "%s\r\n", argv[1]);
    send(sock, query, strlen(query), 0);

    char buf[1024];
    int n;
    while ((n = recv(sock, buf, sizeof(buf)-1, 0)) > 0) {
        buf[n] = '\0';
        printf("%s", buf);
    }
    close(sock);
    return 0;
}
```